

MICROSAR Classic vMem_30_Traveo2Cyp01

Technical Reference

Version 3.06.00

Contents

1	Introduction	5
1.1	Architecture Overview	5
1.2	Component Interfaces	5
2	Functional Description	6
2.1	Features	6
2.1.1	Additions/ Extensions	6
2.1.1.1	SetDualBankMode	6
2.1.1.2	PerformMemorySwap	6
2.1.1.3	Read-Only mode	6
2.1.2	Limitations	7
2.1.2.1	Memory Instances	7
2.1.2.2	ECC Error detection	7
2.1.2.3	RAM alignment	7
2.1.2.4	Write buffer location	7
2.2	Initialization	7
2.3	Main Functions	7
2.4	IPC	8
2.5	System calls	8
2.6	Error Handling	9
2.6.1	Development Error Reporting	9
2.6.2	Handling of non-correctable ECC errors	9
2.7	Hardware Software Interface	9
2.7.1	Flash Controller (FLASHC)	9
2.7.2	Inter Processor Communication (IPC)	10
2.7.3	Fault (FAULT)	10
3	Integration	11
3.1	Embedded Implementation	11
3.2	Header/Footer sections	11
3.3	Critical Sections	12
3.4	Error Reporting Det	12
3.5	Write buffer address alignment	12
3.6	Write buffer location	12
3.7	Execution on different cores	12
3.7.1	Execution on HSM (CM0+) core	13
3.7.2	Execution on CM7 or CM4 core	14
3.8	Dependencies on SW modules	14

3.8.1	OS	14
4	API Description	16
4.1	Type Definitions	16
4.2	Services provided by the vMem_30_Traveo2Cyp01	16
4.2.1	vMem_30_Traveo2Cyp01_SetDualBankMode	16
4.2.2	vMem_30_Traveo2Cyp01_PerformMemorySwap	17
4.3	Required Functions	18
5	Configuration	19
5.1	Execution on CM7 or CM4 core	19
5.2	Execution on HSM (CM0+) core	19
5.2.1	MICROSAR vLinkGen Configuration	19
5.2.2	MICROSAR Os Interrupt Configuration	21
6	Glossary and Abbreviations	24
6.1	Glossary	24
6.2	Abbreviations	24
7	Contact	25

Document Information

Reference Documents

No.	Source	Title	Version
[1]	AUTOSAR	AUTOSAR_SWS_DevelopmentErrorTracer.pdf	3.2.0
[2]	Vector	TechnicalReference_vMem.pdf	see delivery
[3]	Cypress	Traveo II Automotive Body Controller High Family Architecture Technical Reference Manual Document No. 002-24401	Rev. *A
[4]	Cypress	Traveo II Automotive Body Controller High Registers Technical Reference Manual Document No. 002-24402	Rev. *A
[5]	Vector	TechnicalReference_vMemAccM.pdf	7.0.x
[6]	Vector	TechnicalReference_Os.pdf	see delivery
[7]	Vector	AN-ISC-8-1258_vMem_TraveoII_ECC_Handling.pdf	1.x.x
[8]	Infineon	Flash driver user guide TRAVEO T2G family	002-23407 Rev. *T



Caution

We have configured the programs in accordance with your specifications in the questionnaire. Whereas the programs do support other configurations than the one specified in your questionnaire, Vector's release of the programs delivered to your company is expressly restricted to the configuration you have specified in the questionnaire.

1 Introduction

This document describes the functionality, integration, API and configuration of the MICROSAR BSW module vMem_30_Traveo2Cyp01.

Supported AUTOSAR Release	4
Supported Configuration Variants	pre-compile
Vendor ID	VMEM_30_TRAVEO2CYP01_VENDOR_ID (30 decimal = Vector-Informatik according to HIS)
Module ID	VMEM_30_TRAVEO2CYP01_MODULE_ID (255 decimal according to [5])

This document supplements the general vMem driver technical reference (see [2]). The vMem_30_Traveo2Cyp01 driver was tested on hardware with the Cypress Body High CYT4BF8CDAES and Cypress Body Entry CYT2B78CABES.

1.1 Architecture Overview

Refer to [2].

1.2 Component Interfaces

Refer to [2].

2 Functional Description

2.1 Features

2.1.1 Additions/ Extensions

For the standard functionality, which is provided by all vMem drivers, see [2]. The following extended hardware specific features are provided beyond the vMem standard features:

Features Provided Beyond the vMem Standard
SetDualBankMode
PerformMemorySwap
Read-Only mode support (configurable)

Table 2.1 Features provided beyond the vMem standard

2.1.1.1 SetDualBankMode

When the service `vMem_30_Traveo2Cyp01_SetDualBankMode()` is called, the `vMem_30_Traveo2Cyp01` will set the dual bank mode (see [3]) active for the requested flash memory type by setting the *BANK_MODE* bit of the *FLASHC_FLASH_CTL* register.

2.1.1.2 PerformMemorySwap

When the service `vMem_30_Traveo2Cyp01_PerformMemorySwap()` is called, the `vMem_30_Traveo2Cyp01` will swap the mapping from 'Mapping A' to 'Mapping B' or reversed for the requested flash memory type by setting the *MAP* bit of the *FLASHC_FLASH_CTL* register (see [3]).

Dual Bank Mode must be active for the requested flash memory type when calling this service, otherwise `E_NOT_OK` will be returned.

2.1.1.3 Read-Only mode

The `vMem_30_Traveo2Cyp01` supports a read only mode. In the Read-Only mode, the `vMem_30_Traveo2Cyp01` allows and processes only the `vMem_30_Traveo2Cyp01_Read()` and `vMem_30_Traveo2Cyp01_IsBlank()` services, i.e. the services which do not modify non-volatile memory. The services which do modify non-volatile memory like `vMem_30_Traveo2Cyp01_Write()`, `vMem_30_Traveo2Cyp01_Erase()`, `vMem_30_Traveo2Cyp01_SetDualBankMode()` and `vMem_30_Traveo2Cyp01_PerformMemorySwap()` are not supported and rejected with `E_NOT_OK`.

To enable the Read-Only mode, the value of `/MICROSAR/vMem_30_Traveo2Cyp01/vMem/vMemGeneral/vMemReadOnlyMode` must be set to true.



Note

The `vMem_30_Tx3xxInf01` behaves normally (full-mode) in the standard or default scenario.

2.1.2 Limitations

2.1.2.1 Memory Instances

The vMem_30_Traveo2Cyp01 manages only one memory instance. Therefore, the instance identifier, which is passed as a parameter in all services, is not used by the vMem_30_Traveo2Cyp01, except for parameter checks.

2.1.2.2 ECC Error detection

The vMem_30_Traveo2Cyp01 does not evaluate the ECC error status and does not return the results `VMEM_READ_CORRECTED_ERRORS` and `VMEM_READ_UNCORRECTABLE_ERRORS`.

2.1.2.3 RAM alignment

The address of the buffer used for write operations has to be 32-bit (4-byte) aligned.

2.1.2.4 Write buffer location

The address of the buffer used for write operations must be located in the SRAM. A flash address is not supported by the hardware. This must be ensured by the environment, because the vMem does not take care of the write buffer location by itself, or even check its correctness.

2.2 Initialization

Refer to [2].

2.3 Main Functions

The vMem_30_Traveo2Cyp01 has one central processing function `vMem_30_Traveo2Cyp01_MainFunction()`, which is called via `vMemAccM`.

When an asynchronous request is accepted, its processing will be handled within the `vMem_30_Traveo2Cyp01_MainFunction()`. The vMem_30_Traveo2Cyp01 accepts at most one asynchronous job request at a time. The following are the asynchronous API services:

```
> vMem_30_Traveo2Cyp01_Write()
> vMem_30_Traveo2Cyp01_Erase()
> vMem_30_Traveo2Cyp01_IsBlank()
```



Note

The `vMem_30_Traveo2Cyp01_Read` job is performed synchronously as memory-mapped read.

2.4 IPC

The driver uses the Inter Processor Communication to perform flash operations. These operations are implemented as system calls. A dedicated IPC structure (mailbox) is associated with each CPU core. The IPC structure used depends on which core executes the driver. If acquisition of the IPC structure fails, the driver retries a configured number of times, then reports a failed job. Similarly, for the IPC interrupt structures, a dedicated structure is associated with each CPU core. The used resources are summarized as follows:

- > IPC interrupt structure 0 for notifying CM0+ of the System Call.
- > IPC structure 0, 1 or 2 for invoking the System Call from CM0+, CM4 or CM7
- > IPC interrupt structure 2 (default) for notifying CM0+, CM4 or CM7 of completion of the System Call

The following diagram illustrates the System Call interface using the IPC (see [3]):

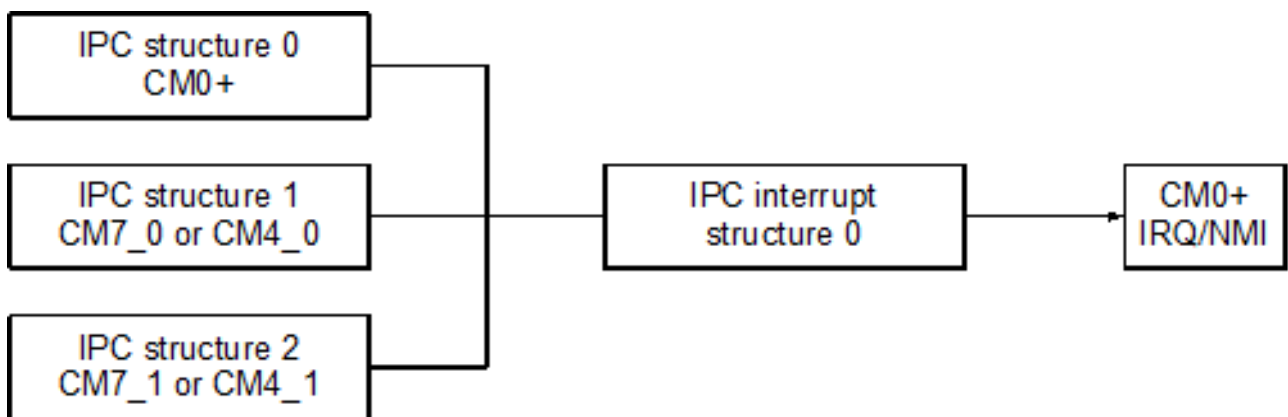


Figure 2.1 System Call interface using the IPC



Caution

The IPC peripheral acts as a shared resource. Due to the fact that there is no software standard (e.g. AUTOSAR) the handling of this shared resource may impact your software integration. More details see the following chapters e.g. 3.6.

2.5 System calls

The IRQ0 exception is used to trigger a System Call. The System Calls are used to implement the flash programming operations and are executed inside CM0+. The System Calls make use of IPC to initiate an IRQ to CM0+. The SROM API Library can be called via a System Call. Therefore, a pointer to the System Call opcode and arguments must be written to the IPC mailbox. The triggered System Call is serviced by the CM0+ core IRQ0 exception handler which resides in the SROM API Library. The driver makes use of the following System Calls:

Name	Opcode	Description
ProgramRow	0x06	Programs the addressed FLASH page
EraseSector	0x14	Erases the addressed FLASH sector
BlankCheck	0x2A	Performs blank check on eCT work flash memory.

Table 2.2 System Calls used by the vMem_30_Traveo2Cyp01

2.6 Error Handling

2.6.1 Development Error Reporting

By default, development errors are reported to the DET using the service `Det_ReportError()` as specified in [2], if development error reporting is enabled.

If another module is used for development error reporting, the function prototype for reporting the error can be configured by the integrator, but must have the same signature as the service `Det_ReportError()`.

The reported vMem_30_Traveo2Cyp01 ID is 255.

Note that the standard services of a vMem driver and their corresponding service IDs are described in [2]. The following service IDs identify the extended hardware specific services which are described in 4.2. The following table presents the service IDs and the related services:

Service ID	Service
0x11	<code>vMem_30_Traveo2Cyp01_SetDualBankMode()</code>
0x12	<code>vMem_30_Traveo2Cyp01_PerformMemorySwap()</code>

Table 2.3 Service IDs



Note

All used error codes reported to DET are described in the corresponding table in [2], chapter Error Handling.

2.6.2 Handling of non-correctable ECC errors

As mentioned in 2.1.4.2, vMem_30_Traveo2Cyp01 does not evaluate the ECC error status. However, ECC checking for flash work interface is enabled during initialization. Non-correctable ECC errors will lead to bus errors and can be suppressed with the configuration switch `vMem/vMemGeneral/vMemSuppressEccErrors`. Please refer to [7] for a detailed description of ECC handling with vMem_30_Traveo2Cyp01.

2.7 Hardware Software Interface

2.7.1 Flash Controller (FLASHC)

The vMem_30_Traveo2Cyp01 accesses the registers of the FLASHC to configure the flash, enabling the dual bank mode, perform memory swap, check the flash status, flash operation

and error status. The registers are described in see [4].

2.7.2 Inter Processor Communication (IPC)

The vMem_30_Traveo2Cyp01 uses the dedicated IPC structure to transfer the data for the System Calls. The reserved IPC interrupt structures are used to inform the sender and receiver about the System Call. For all structures read and write accesses are performed.

2.7.3 Fault (FAULT)

The vMem_30_Traveo2Cyp01 reads the FAULT structs to retrieve the status of the ECC errors. This feature is currently not supported.

3 Integration

This chapter gives necessary information for the integration of the MICROSAR vMem_30_Traveo2Cyp01 into an application environment of an ECU.

3.1 Embedded Implementation

The delivery of the vMem_30_Traveo2Cyp01 contains the files which are described in the corresponding chapters in [2]. In addition, the vMem_30_Traveo2Cyp01 contains the following static file(s).

File Name	Description
vMem_30_Traveo2Cyp01_LL_FlashMgtLib.h	This file declares the interface of the Flash Management Library.
vMem_30_Traveo2Cyp01_LL_FlashMgtLib.c	This file contains the Flash Management Library related implementation.
vMem_30_Traveo2Cyp01_LL_lpc.h	This file declares the interface of the Inter Processor Communication related part.
vMem_30_Traveo2Cyp01_LL_lpc.c	This file contains the Inter Processor Communication related implementation.
vMem_30_Traveo2Cyp01_LL_RegAccess_Int.h	This header file contains the functions for the register read, write access.
vMem_30_Traveo2Cyp01_LL_Regs.h	This header file contains the macro definitions of all the used registers and the register bit-masks.
vMem_30_Traveo2Cyp01_Extended_Func.h	This header file contains the types and declarations of the extended hardware specific functionality of the vMem. It should be included by every user, who wants to call a hardware specific service.
vMem_30_Traveo2Cyp01_LL_DetChecks.h	This file contains DetChecks for low level APIs.

Table 3.1 Additional static files

3.2 Header/Footer sections

For the downloadable driver support, the vMem_30_Traveo2Cyp01 provides special sections which contain the header and footer data (FPT, unique identifier, address of header and footer etc). These sections must be mapped appropriately for the correct verification and usage of vMem_30_Traveo2Cyp01 in the flash bootloader context.

- > VMEM_30_TRAVEO2CYP01_START_SEC_HEADER_CONST_UNSPECIFIED/ VMEM_30_TRAVEO2CYP01_STOP_SEC_HEADER_CONST_UNSPECIFIED
- > VMEM_30_TRAVEO2CYP01_START_SEC_FOOTER_CONST_32BIT/ VMEM_30_TRAVEO2CYP01_STOP_SEC_FOOTER_CONST_32BIT

**Note**

The mapping of these sections is specifically relevant for the FBL- downloadable driver use-case. For all other use-cases, they should be treated in the same way as the other memory sections of the vMem_30_Traveo2Cyp01.

3.3 Critical Sections

To protect critical code against interruptions the vMem_30_Traveo2Cyp01 uses the following exclusive area:

> VMEM_30_TRAVEO2CYP01_EXCLUSIVE_AREA_0

The exclusive area covers only some statements which ensures short runtime. The critical section is used vMem_30_Traveo2Cyp01 internally and does not cover calls to other modules. vMem_30_Traveo2Cyp01 does not nest critical sections.

3.4 Error Reporting Det

The module has these properties used for the error reporting to the Det.

> VMEM_30_TRAVEO2CYP01_MODULE_ID = 255

> VMEM_30_TRAVEO2CYP01_VENDOR_ID = 30

3.5 Write buffer address alignment

The user of the vMem_30_Traveo2Cyp01 must ensure that the address of the write buffer (with user data) passed during a write operation is 32-Bit aligned.

3.6 Write buffer location

The address of the passed write buffer (user data) must be a valid SRAM address. A Flash address is not supported and will lead to hardware exceptions.

3.7 Execution on different cores

The driver is designed to run on all cores of the Traveo II without major adjustments. Therefore, it was decided to use the System Call mechanism for every core to obtain a simple software architecture.

**Caution**

The shared IPC resources must be considered during the integration of the vMem_30_Traveo2Cyp01. The driver directly accesses the IPC registers to perform a System Call. Therefore, any further instance acting on the IPC resource may not influence and impact our driver e.g. any proprietary IPC library functionalities, which perform supervision of the IPC module!

For the driver to work correctly, the integrator must ensure a good distribution of the shared resources by selecting the call sequence of the `MainFunction()` sensibly. The goal is to minimize or eliminate the occurrence of IPC get lock timeouts.

Other instances are allowed to use the same shared IPC structures as long as they adhere in software to the lock and release rules described in [3]. These points are critical to proper functioning of the vMem:

- > IPC structures are not allowed to be used by any other instance if the structure is already locked.
- > If an instance is using an IPC structure it always has to successfully acquire the lock of that same ipc structure, so other instances can see that it is in use.
- > If an instance does not use an IPC structure anymore it has to make sure that this IPC structure is released eventually. So other instances can acquire the lock successfully afterwards.

The vMem adheres to this rules at any time and expects other instances to do so as well. If the IPC structure takes too long to be released again, the vMem will eventually fail with a timeout.



Caution

The fls driver by Infineon does not adhere to these rules but instead uses a different approach which is called "IPC based safety mechanism" in Infineons fls user guide (see [8]). Since the vMem does not implement this approach, the vMem does currently not support to run in parallel with Infineons fls driver.

3.7.1 Execution on HSM (CM0+) core

As previously mentioned, the System Call mechanism is also used for CM0+. A working System Call is a prerequisite for the proper operation of the driver. To run the driver directly on the HSM core you must handle IRQ0 and IRQ1 which are used by the System Call in a precisely specified manner.

- > CPU IRQ0 is the entry point for the System Call.
- > Software IRQ1 is triggered by the IRQ0 handler and ensures the correct protection context when leaving the handler.

For a working System Call ensure the following:

1. Ensure that your current vector table IRQ0 and IRQ1 entries match those of the SROM interrupt vector table (note that IRQ0 and IRQ1 can no longer be used for other purposes by the application if system calls shall be processed).
The SROM table handler addresses can be found at absolute addresses 0x40 (IRQ0) and 0x44 (IRQ1).
2. Set the NVIC IRQ0 priority to 1 and IRQ1 priority to 0 (default).
3. Enable IRQ0 and IRQ1 in the NVIC.
4. Clear PRIMASK.

**Note**

If you are using a MICROSAR Os and vLinkGen please see 5.2.2 and 5.2.1 on how to configure the interrupts and vector table addresses.

3.7.2 Execution on CM7 or CM4 core

For execution on CM7 or CM4 core no interrupts are needed. The IPC interrupt structure flags are polled during the `MainFunction()` cycle. The flashed software on core CM0+ must handle the System Calls correctly and process the flash operations.

3.8 Dependencies on SW modules

3.8.1 OS

Depending on the configuration of the parameter `vMem_30_Traveo2Cyp01_UsePeripheralAccessApi` the memory mapped registers are either accessed directly or via the APIs offered by the OS.

The OS services for the privileged access to protected registers which are called by the vMem must be provided by the Operating System. Please refer to the Technical Reference [6] for further information.

If the peripheral access API shall be used, the following Os Peripheral Regions must be created in the OS module in DaVinci Configurator. For the MICROSAR OS all Peripheral Regions will be added automatically by the DaVinci Configurator (Call the solving action “OsPeripheralRegion is missing”).

Flash Controller (FLASHC)

Parameter	Value
Shortname	OS_MEM_AREA_VMEM_30_TRAVEO2CYP01_REG_FLASHC
Peripheral Region Start Address	0x40240000
Peripheral Region End Address	0x4025FFFF

Table 3.2 Peripheral Region FLASHC

Inter Processor Communication (IPC)

Parameter	Value
Shortname	OS_MEM_AREA_VMEM_30_TRAVEO2CYP01_REG_IPC
Peripheral Region Start Address	0x40220000
Peripheral Region End Address	0x4022FFFF

Table 3.3 Peripheral Region IPC

Fault (FAULT)

Parameter	Value
Shortname	OS_MEM_AREA_VMEM_30_TRAVEO2CYP01_REG_FAULT
Peripheral Region Start Address	0x40210000
Peripheral Region End Address	0x4021FFFF

Table 3.4 Peripheral Region FAULT

4 API Description

4.1 Type Definitions

The types needed for the extended hardware specific functionality defined by the vMem_30_Traveo2Cyp01 are described in this chapter. For the definition of the standard vMem types see [2].

Type Name	C-Type	Description	Value Range
vMem_30_Traveo2Cyp01_HWSpecificFuncID	enum	Index of hardware specific functionality in function pointer table.	VMEM_30_TRAVEO2CYP01_SETDUALBANKMODE Function index of the hardware specific service vMem_30_Traveo2Cyp01_SetDualBankMode(). VMEM_30_TRAVEO2CYP01_PERFORMMEMORYSWAP Function index of the hardware specific service vMem_30_Traveo2Cyp01_PerformMemorySwap().
vMem_30_Traveo2Cyp01_FlashTypeType	enum	Describes the requested flash type.	VMEM_30_TRAVEO2CYP01_FLASH_TYPE_CODE = 0 VMEM_30_TRAVEO2CYP01_FLASH_TYPE_WORK = 1

Table 4.1 Type definitions of extended hardware specific functionality

4.2 Services provided by the vMem_30_Traveo2Cyp01

The APIs for the extended hardware specific functionality defined by the vMem_30_Traveo2Cyp01 are described in this chapter. For the APIs of the standard vMem functionality see [2].

4.2.1 vMem_30_Traveo2Cyp01_SetDualBankMode

Prototype

```
Std_ReturnType vMem_30_Traveo2Cyp01_SetDualBankMode (  
vMem_30_Traveo2Cyp01_InstanceIdType InstanceId,  
vMem_30_Traveo2Cyp01_DataPtrType ReqSpecificData,  
uint32 SizeOfData)
```

Parameter

InstancelId [in]	ID of the related vMem instance.
ReqSpecificData [in,out]	Pointer to a variable of type vMem_30_Traveo2Cyp01_FlashTypeType. Contains the information for which flash type the dual bank mode shall be set active.
SizeOfData [in]	Not needed for this request.

Return code	
Std_ReturnType	E_OK request accepted, E_NOT_OK otherwise.

Functional Description
Sets the dual bank mode active for the requested flash type. Note: This service must be called via the HwSpecificReq API of the vMemAccM. The corresponding function index is VMEM_30_TRAVEO2CYP01_SETDUALBANKMODE. Via this service, the dual bank mode (see [3]) will be activated for the requested flash type by setting the <i>BANK_MODE</i> bit of the <i>FLASHC_FLASH_CTL</i> register.

Particularities and Limitations
None

Call context
> TASK > This function is synchronous. > This function is reentrant.

4.2.2 vMem_30_Traveo2Cyp01_PerformMemorySwap

Prototype
<pre>Std_ReturnType vMem_30_Traveo2Cyp01_PerformMemorySwap (vMem_30_Traveo2Cyp01_InstanceIdType InstanceId, vMem_30_Traveo2Cyp01_DataPtrType ReqSpecificData, uint32 SizeOfData)</pre>

Parameter	
Instanceld [in]	ID of the related vMem instance.
ReqSpecificData [in,out]	Pointer to a variable of type vMem_30_Traveo2Cyp01_FlashTypeType. Contains the information for which flash type the memory swap shall be performed.
SizeOfData [in]	Not needed for this request.

Return code	
Std_ReturnType	E_OK request accepted, E_NOT_OK otherwise.

Functional Description
Performs a memory swap for the requested flash type. Note: This service must be called via the HwSpecificReq API of the vMemAccM. The corresponding function index is VMEM_30_TRAVEO2CYP01_PERFORMMEMORYSWAP. Via this service, the mapping is swapped from 'Mapping A' to 'Mapping B' or reversed for the requested flash type by setting the <i>MAP</i> bit of the <i>FLASHC_FLASH_CTL</i> register (see [3]). Dual Bank Mode must be active for the requested flash when calling this service, otherwise E_NOT_OK will be returned.

Particularities and Limitations
None

Call context
> TASK > This function is synchronous. > This function is reentrant.

4.3 Required Functions

In addition to the services described in [2] the vMem_30_Traveo2Cyp01 uses services provided by other components as in the following table. For details about prototype and functionality refer to the documentation of the providing component.

Module	API
Os (optional)	osReadPeripeheral32() osWritePeripheral32()

Table 4.14 Required Functions

5 Configuration

Information on the configuration in general can be found in [2].

For details on configuration options refer to the module-specific description that is shown in DaVinci Configurator Classic.

5.1 Execution on CM7 or CM4 core

For the execution on the CM7 or CM4 core only the correct IPC Interrupt Structure Id (vMemGeneral/vMemIPCStructureId) must be configured. See Figure 2-1 for more information.

5.2 Execution on HSM (CM0+) core

For the execution on the HSM core also the correct IPC Interrupt Structure Id (vMemGeneral/vMemIPCStructureId) must be configured. See Figure 2-1 for more information. If you are using a MICROSAR Os and the vLinkGen, read the next chapters attentively, because these are the dependencies that must also be configured properly for a working vMem_30_Traveo2Cyp01.

The following configuration notes apply only when the CM0+ is in Handler mode (see [3]). Other modes have not been tested.

5.2.1 MICROSAR vLinkGen Configuration

The configured handler in the Os doesn't have to be implemented by your own. You must define these labels directly to the SROM vector table handlers. The following table shows the vector table handlers for different derivatives.

Name	Body Entry address	Body High address
_lpc0lsr	0x49	0x49
_lpc1lsr	0x241 Rev. A or 0x22D Rev. B	0x265 Rev. A or 0x2C1 Rev. B

Table 5.1 IPC handler addresses examples



Note

The addresses to the vector table handlers (IRQ0 and IRQ1 service routines) may change from hardware revision to hardware revision. The correct addresses for your hardware can be found at absolute addresses 0x40 (IRQ0) and 0x44 (IRQ1) in the memory view. Take these values for the configuration of the vLinkGen. Otherwise, the System Call mechanism will not work properly!

In the following you will see an example configuration of the vLinkGen LinkerSymbols for the Traveo Body High.

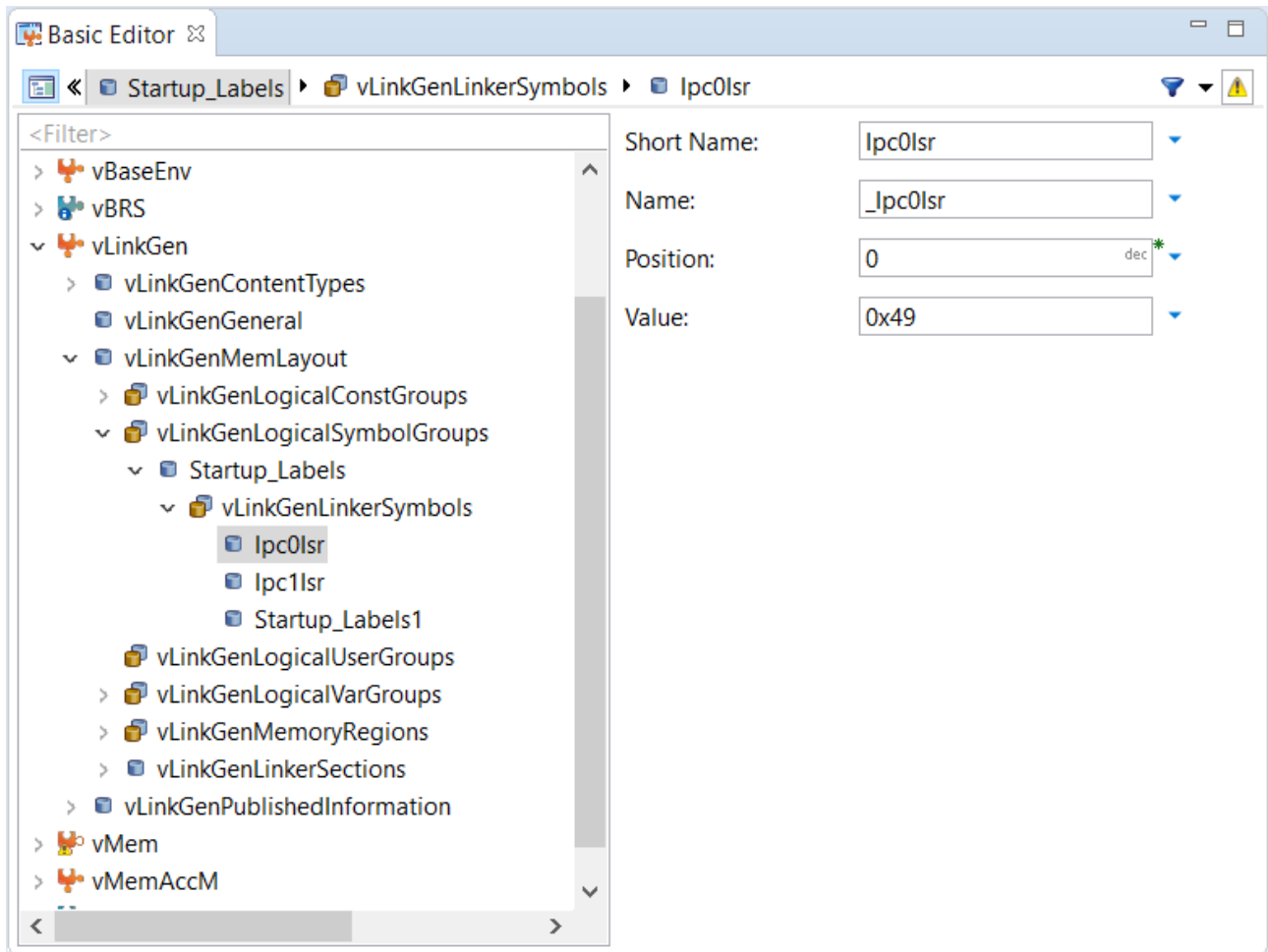


Figure 5.1 Example Ipc0 LinkerSymbol configuration of the vLinkGen

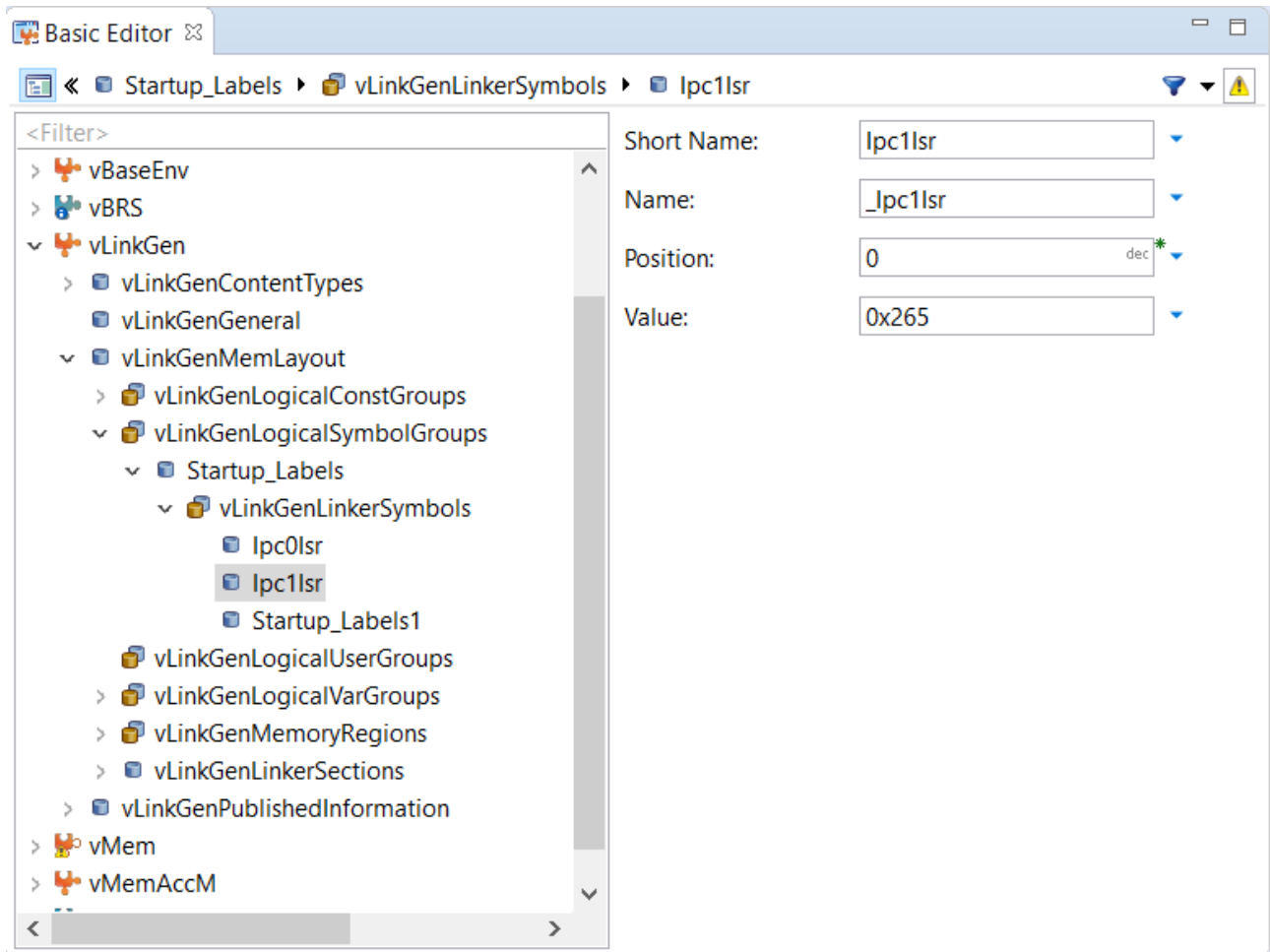


Figure 5.2 Example Ipc1 LinkerSymbol configuration of the vLinkGen

5.2.2 MICROSAR Os Interrupt Configuration

The following Interrupts IPC0 and IPC1 must be registered by the Microsar Os.

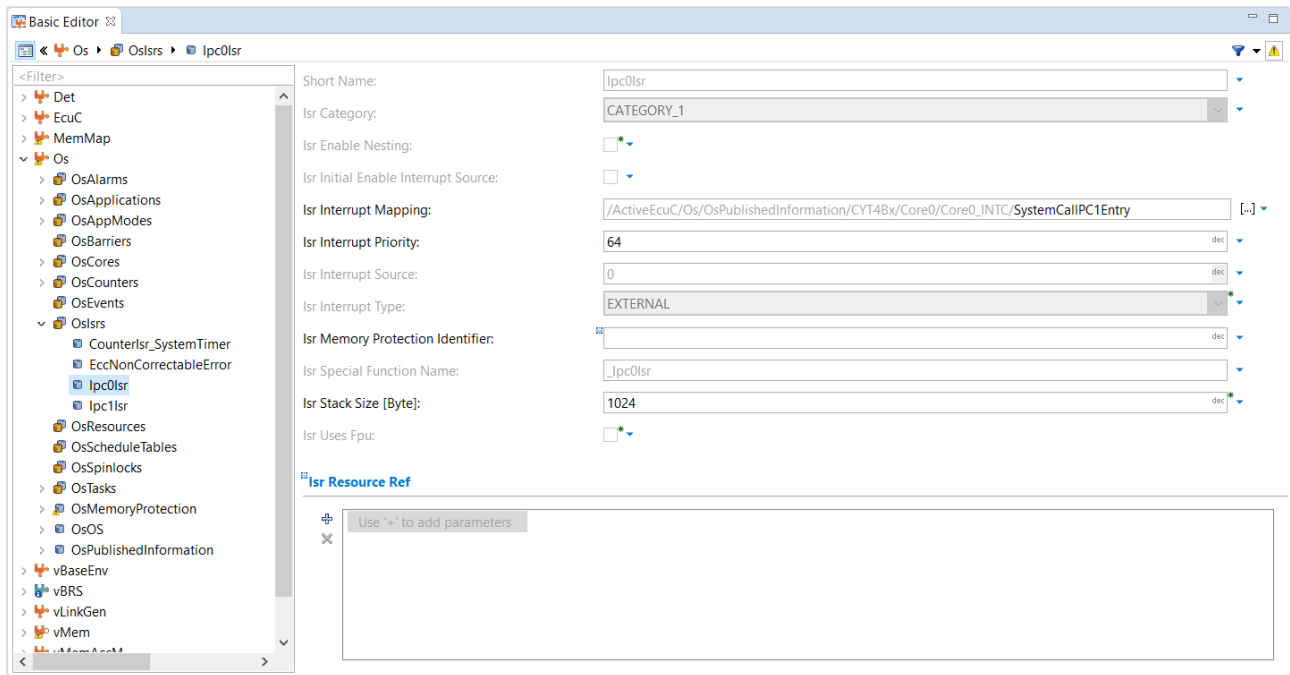


Figure 5.3 Configuration of the IPC0 interrupt in the MICROSAR Os

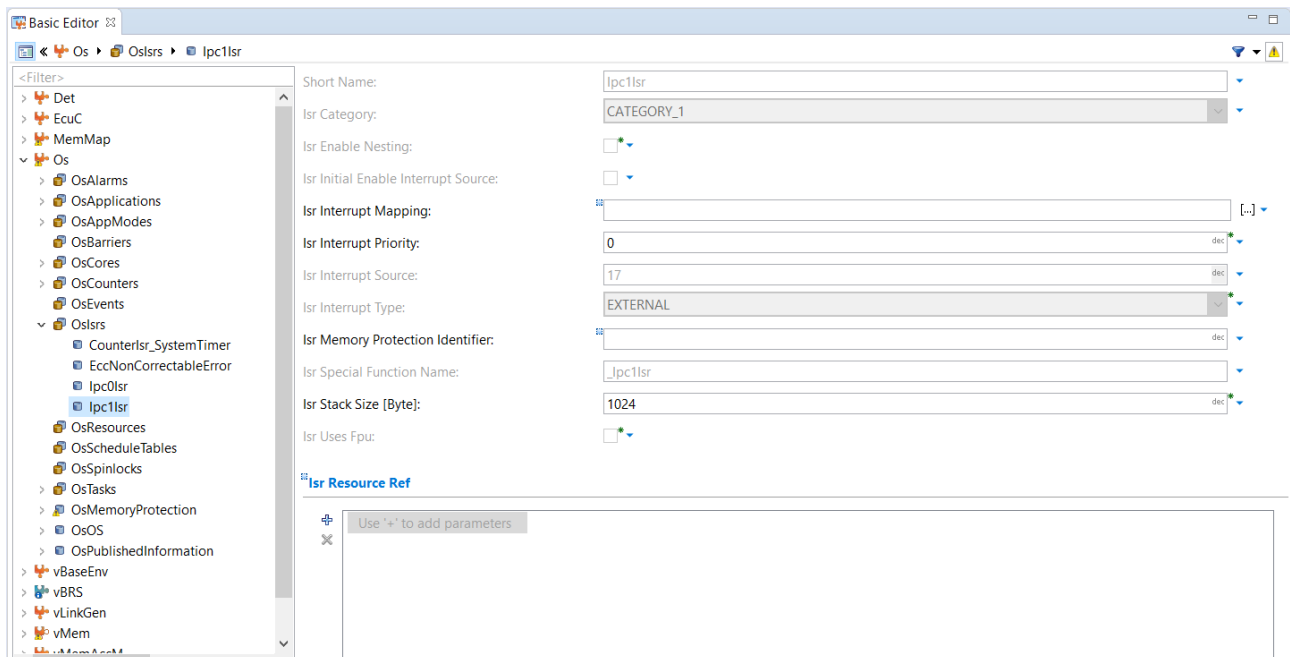


Figure 5.4 Configuration of the IPC1 interrupt in the MICROSAR Os

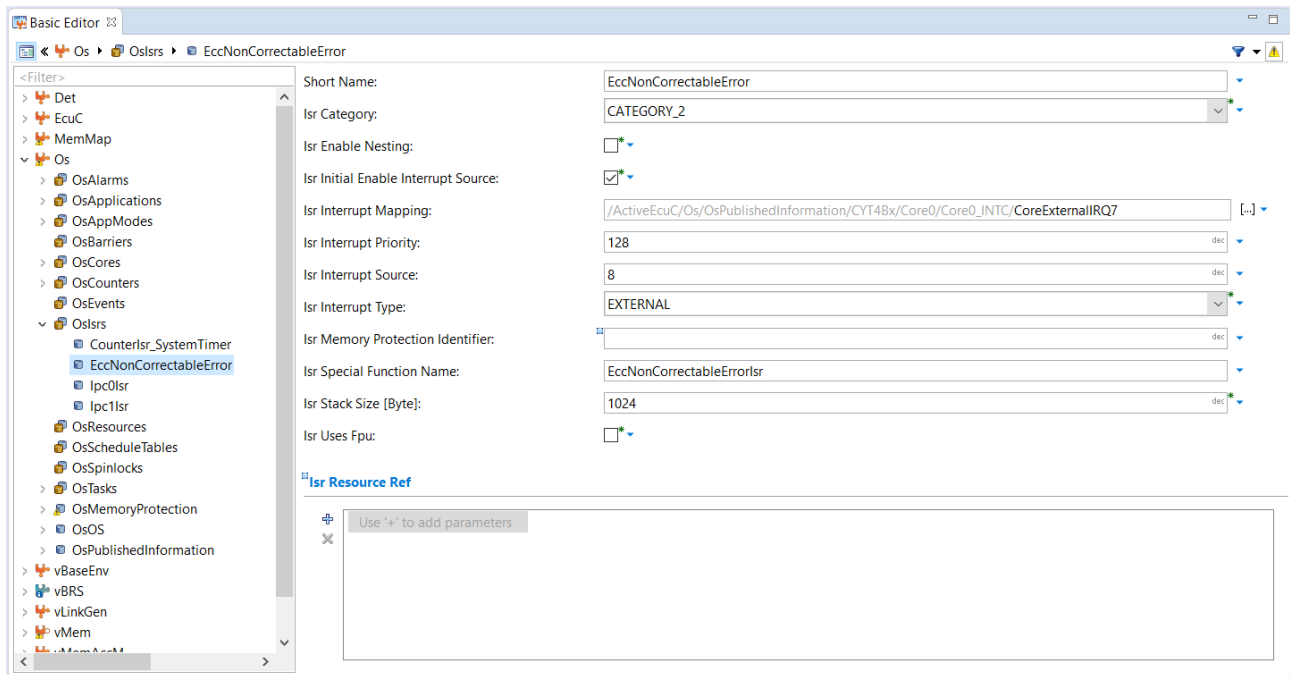


Figure 5.5 Configuration of the ECC interrupt in the MICROSAR Os

6 Glossary and Abbreviations

6.1 Glossary

Term	Description
DaVinci Configurator	Generation tool for MICROSAR Classic modules
vMemAccM	Vector Memory Access Manager: The most likely user of the vMem driver. The vMemAccM partitions and distributes memory access jobs from multiple users to the various vMem drivers.

6.2 Abbreviations

Abbreviation	Description
API	Application Programming Interface
AUTOSAR	Automotive Open System Architecture
BSW	Basis Software
CPU	Central Processing Unit
DEM	Diagnostic Event Manager
DET	Development Error Tracer
ECC	Error Correcting Code
eCT	embedded Charge Trap
ECU	Electronic Control Unit
FLASHC	Flash Controller
HIS	Hersteller Initiative Software
HSM	Hardware Security Module
IPC	Inter Processor Communication
IRQ	Interrupt Request
ISR	Interrupt Service Routine
NVIC	Nested Vector Interrupt Controller
MICROSAR	Microcontroller Open System Architecture (the Vector AUTOSAR solution)
Os	Operating System
RTE	Runtime Environment
SRAM	Static Random Access Memory
SWC	Software Component
SWS	Software Specification

Table 6.2 [Abbreviations]

7 Contact

Visit our website for more information on

- > News
- > Products
- > Demo software
- > Support
- > Training data
- > Addresses

www.vector.com